Introduction aux modèles génératifs

Compte rendu détaillé — trois parties expérimentales

Binôme : Amjad AAYD et [redacted]
[redacted] · Groupe : à compléter
Enseignant : [redacted]
Date de réalisation : 6 octobre 2026
Sujet : série de travaux pratiques du 20 avril 2026, huit pages

Objectif général

Relier estimation de distributions et génération de nouvelles observations. Le travail étudie des lois explicites et leur maximum de vraisemblance, un générateur affine d'images construit à partir d'une covariance empirique, puis des modèles autorégressifs à mémoire limitée. Les trois parties du document sont traitées dans trois notebooks distincts et exécutés.

Partie	Expériences réalisées
1 — Probabilités	Bernoulli directe/sigmoïde ; gaussiennes 1D et 3D ; GMM 3D avec EM manuel ; GMM sinusoïdal 2D.
2 — Images	Moyenne/covariance de chiffres 8×8 ; eigen-images ; génération r=10,25,50 ; variation des coordonnées latentes.
3 — Séquences	Corpus français ; dictionnaires de bigrammes ; ordre 2 ; corpus symbolique ; lissage ; NLL ; quatre lots de 20 séquences.

Principaux résultats

Bernoulli retrouve le MLE 0,696 depuis θ=0,5. La gaussienne univariée estime μ≈1,95969 et σ≈1,51923. La covariance 3D optimisée concorde avec le MLE à la précision numérique. Le GMM retrouve les trois composantes après alignement. Pour les images, 25 composantes conservent 93.36 % de la variance. Sur le corpus symbolique, l'ordre 2 non lissé réduit la NLL d'entraînement, sans prouver une meilleure généralisation.

Les trois notebooks comportent 17 cellules de code exécutées et 15 figures expérimentales. Le rapport reprend les métriques réellement enregistrées et précise les conventions, contrôles et limites. Il ne remplace pas les notebooks complets.

1. Démarche et environnement

1.1. Estimer puis générer

Un modèle génératif définit une distribution pθ(x). L'estimation recherche des paramètres donnant une vraisemblance élevée aux observations disponibles. La génération tire de nouvelles observations depuis la loi ainsi définie. Les deux opérations sont distinctes : retrouver les paramètres empiriques ne signifie pas reproduire exactement les valeurs vraies à taille finie.

Le maximum de vraisemblance maximise la somme des log pθ(xi). En pratique, les expériences minimisent la NLL moyenne, c'est-à-dire l'opposé de cette somme divisé par N. Les logarithmes sont naturels : les NLL sont exprimées en nats. Dans les séquences, le rapport distingue NLL par séquence et par token.

1.2. Versions effectivement utilisées

Composant	Version
Python	3.12.10
NumPy	2.3.5
PyTorch	2.12.0.dev20260408+cu128
Matplotlib	3.10.8
scikit-learn	1.8.0
SciPy	1.17.1

Un environnement de travail distinct avec system-site-packages réutilise les bibliothèques scientifiques déjà installées, sans les réinstaller ni les modifier. PyTorch est une version de développement préexistante avec build CUDA ; les calculs de ce TP utilisent uniquement le CPU, float64 et deux threads. L'instantané des versions est livré dans versions_environnement.json.

1.3. Reproductibilité et contrôles

Les graines sont 42, 43 et 44 pour les trois parties. Les notebook sont autonomes et leurs figures sont enregistrées. Les contrôles portent sur les valeurs MLE, les contraintes de probabilité et de covariance, la log-vraisemblance EM, les dimensions des tirages, la reconstruction spectrale, les moments du générateur et les distributions de transition.

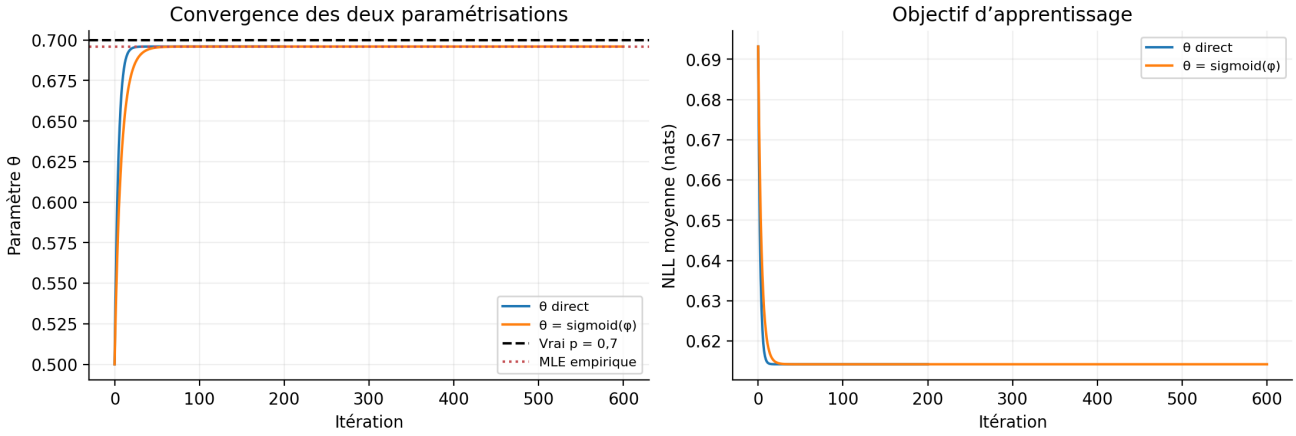
Les choix laissés libres par le sujet sont explicités : paramètres 3D, petit corpus français, jeu digits, sous-ensemble de 1 500 images et rangs 10/25/50. Le bruit sinusoïdal est centré, avec variance 0,1, donc écart-type √0,1. Les comparaisons de qualité des séquences restent descriptives sur quatre exemples d'entraînement.

2. Bernoulli — deux paramétrisations

2.1. Modèle et objectif

Pour $x \in \{0,1\}$, $p(x|\theta)=\theta^x(1-\theta)^{1-x}$. Sur N=1 000 tirages de paramètre vrai 0,7, la NLL moyenne vaut $-\bar{x}\log\theta - (1-\bar{x})\log(1-\theta)$. Le MLE analytique est $\bar{x}$. Le gradient direct est $(\theta-\bar{x})/(\theta(1-\theta))$.

La première expérience démarre à θ=0,5, utilise SGD de pas 0,05 sur 200 mises à jour et maintient θ dans (0,1) par clamp. La seconde démarre à φ=0, donc θ=σ(φ)=0,5, et optimise φ avec un pas 0,5 sur 600 mises à jour. La sigmoïde assure la contrainte sans clamp ; le gradient en φ vaut $\theta-\bar{x}$.



Référence / méthode	Valeur θ
Paramètre vrai	0.700000
MLE analytique	0.696000
Gradient direct	0.696000
Sigmoïde	0.696000

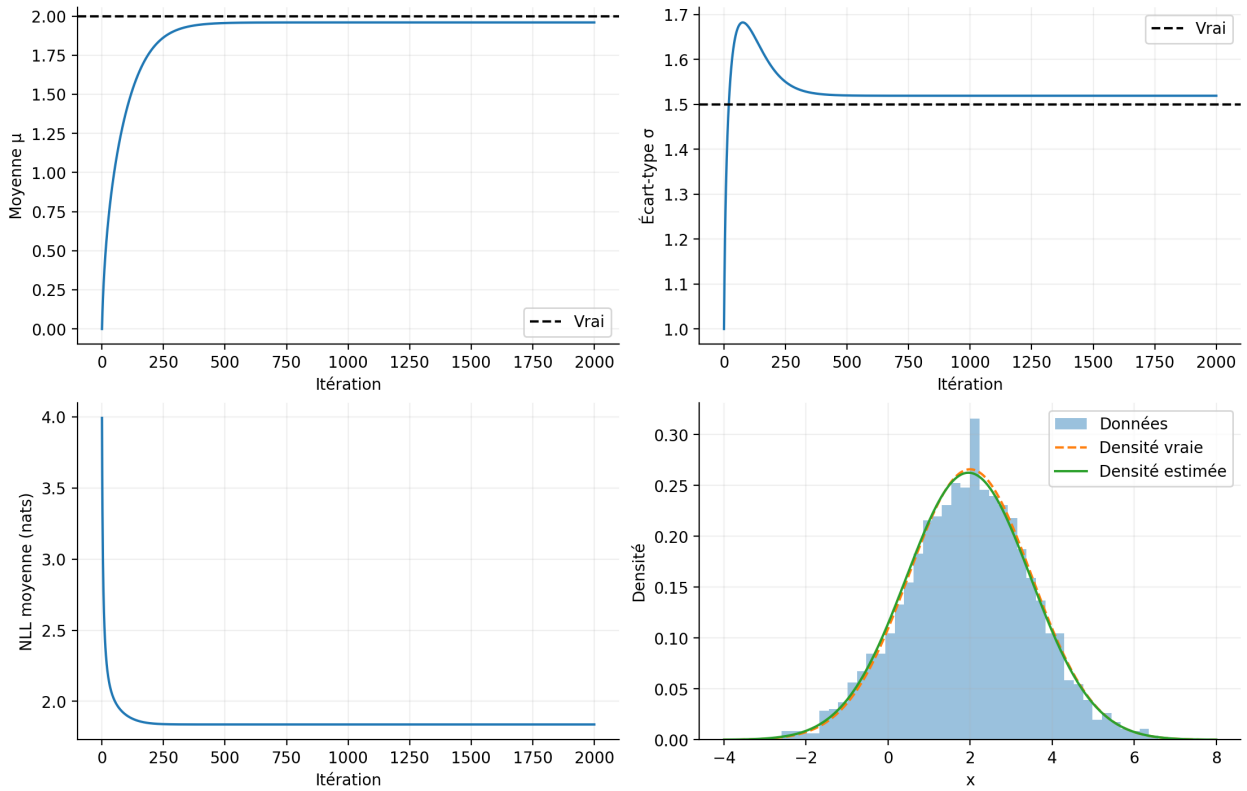
Les deux estimations sont à moins de 10^{-6} du MLE. La NLL finale est 0.614215. L'écart 0,696 contre 0,7 provient de l'échantillon simulé. Le clamp empêche les logarithmes invalides mais doit être choisi avec prudence près des bornes ; la reparamétrisation exprime directement la contrainte dans le modèle.

3. Gaussienne univariée — μ et σ

3.1. NLL et positivité

Les 2 000 données sont tirées de N(2,0 ; 1,5²). La NLL moyenne est log σ + ½log(2π) + moyenne((x−μ)²)/(2σ²). Le paramètre σ=softplus(ρ)+10^{-6} reste positif. μ démarre à zéro et σ à environ 1 ; SGD utilise un pas 0,03 pendant 2 000 mises à jour.

softplus(ρ)=log(1+exp(ρ)) est une transformation vers les valeurs positives. Sa mise en œuvre numérique stable est fournie par PyTorch. Documentation de la fonction.



Paramètre	Vrai	MLE / gradient
μ	2,000000	1.959693
σ	1,500000	1.519230
NLL moyenne	—	1.837142

L'estimation rejoint les références empiriques à moins de 10^{-5}. Pour σ, la variance MLE utilise le diviseur N, sans correction N−1. L'histogramme est normalisé en densité afin de comparer sa forme aux densités vraie et apprise. Les courbes montrent convergence de l'optimisation ; les petits écarts au vrai modèle restent compatibles avec la fluctuation de 2 000 observations.

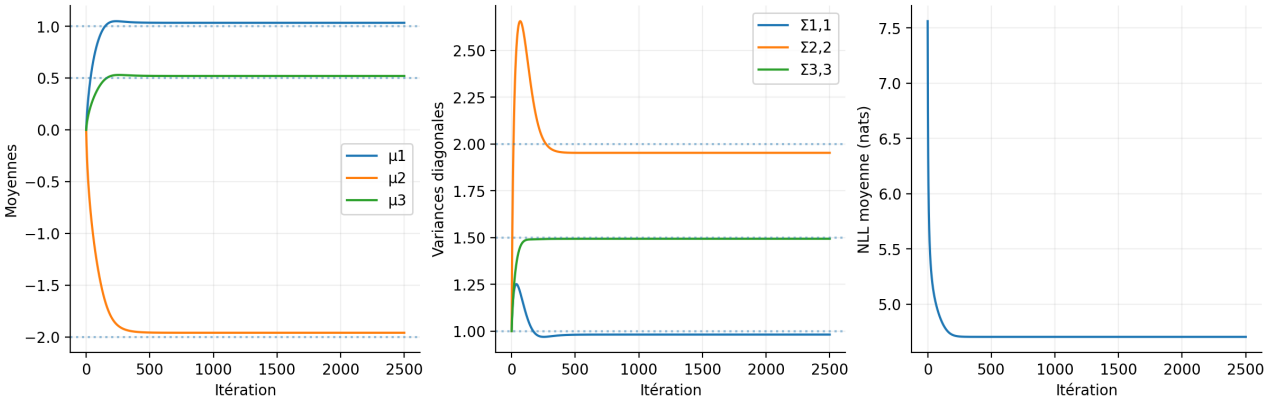
4. Gaussienne 3D — covariance et Cholesky

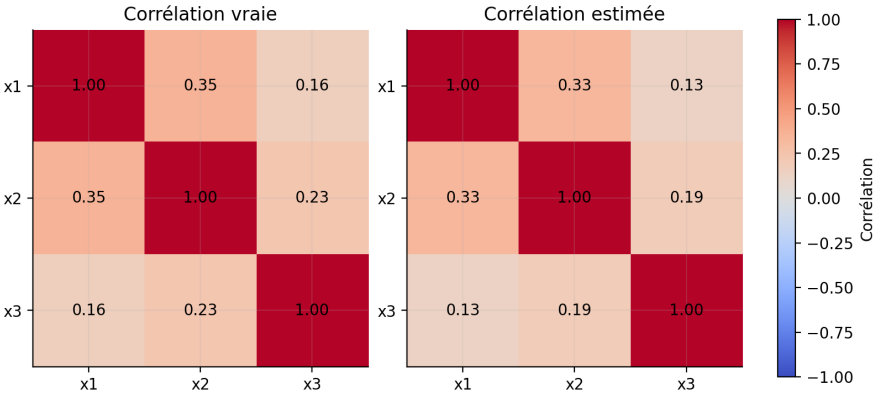
Le modèle choisi a μ=[1.0000 ; -2.0000 ; 0.5000] et une covariance symétrique définie positive. Les 2 000 observations sont estimées par SGD de pas 0,03 sur 2 500 mises à jour. L est triangulaire inférieure ; sa diagonale est softplus+ε, ce qui impose $\Sigma = LL^T$ définie positive.

```
NLL moyenne = (3/2) log(2π) + somme(log(diag(L)))
              + 0.5 * moyenne(|| solve(L, x - mu) ||²)
```

Le calcul utilise une résolution triangulaire, sans inversion explicite de Σ. log|Σ| vaut deux fois la somme des log de la diagonale de L. La covariance empirique de référence est calculée avec le diviseur N.

```
Covariance vraie :
  1.00000   0.50000   0.20000
  0.50000   2.00000   0.40000
  0.20000   0.40000   1.50000
```





Moyenne estimée : [1.0337 ; -1.9603 ; 0.5202].
Erreur maximale de covariance par rapport au MLE empirique : 4.88e-15. Les contrôles confirment des valeurs propres strictement positives et une moyenne proche du MLE à moins de 10^{-4}.

Les heatmaps comparent les corrélations vraies et estimées, sur une échelle commune −1 à +1. Une corrélation normalise la covariance par les écarts-types ; elle permet de distinguer dépendance et simple changement d'échelle. La concordance avec le MLE ne doit pas être confondue avec une égalité exacte à la covariance vraie.

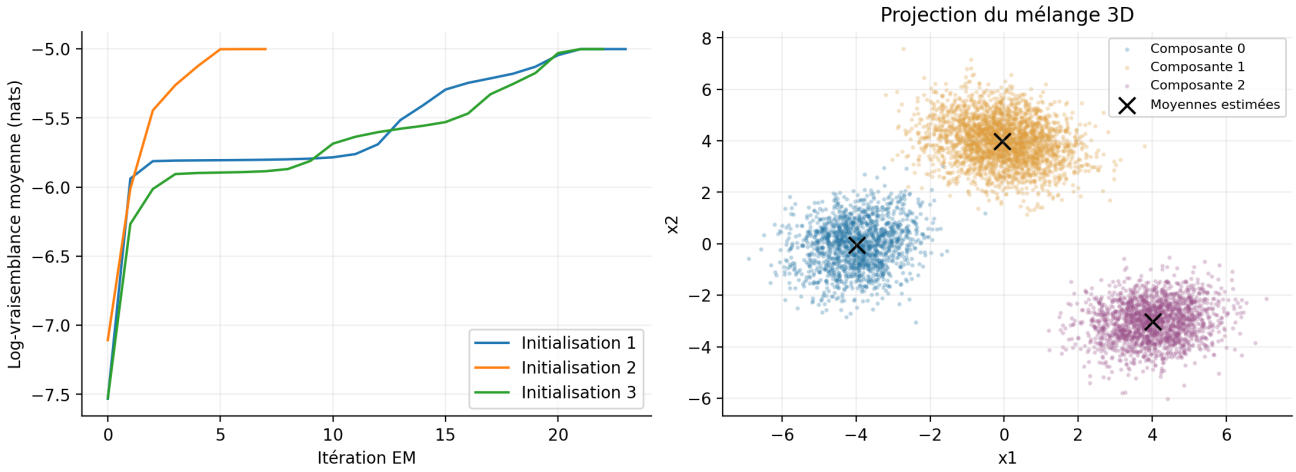
5. GMM 3D — EM manuel et convergence

5.1. Données et étapes EM

N=6 000 observations sont simulées avec K=3 composantes, dimension d=3. Les poids vrais sont [0.2500 ; 0.4500 ; 0.3000]. Les moyennes choisies sont bien séparées afin de rendre l'interprétation des affectations lisible.

E-step : r_{ik} est proportionnel à $\pi_k\, \mathcal{N}(x_i|\mu_k,\Sigma_k)$, puis normalisé sur k. Le calcul passe par des log-densités et logsumexp pour limiter les sous-flux numériques.
M-step : $N_k=\sum_i r_{ik}$; $\pi_k=N_k/N$; $\mu_k=\sum_i r_{ik}\, x_i/N_k$; $\Sigma_k=\sum_i r_{ik}(x_i-\mu_k)(x_i-\mu_k)^T/N_k$. Une régularisation diagonale 10^{-6} évite les singularités numériques.

Les trois essais démarrent avec des poids uniformes, des moyennes tirées aléatoirement parmi les données et des covariances globales régulières. Le critère d'arrêt est un gain moyen absolu de log-vraisemblance inférieur à 10^{-6}, ou 150 mises à jour. L'essai ayant la meilleure valeur finale est conservé.



Le meilleur essai s'arrête après 7 mises à jour. Sa log-vraisemblance totale est -30012.693, soit environ -5.002116 par observation. Le plus petit gain total enregistré vaut 1.107e-07. Les responsabilités somment à un, les poids sont positifs et les covariances restent définies positives.

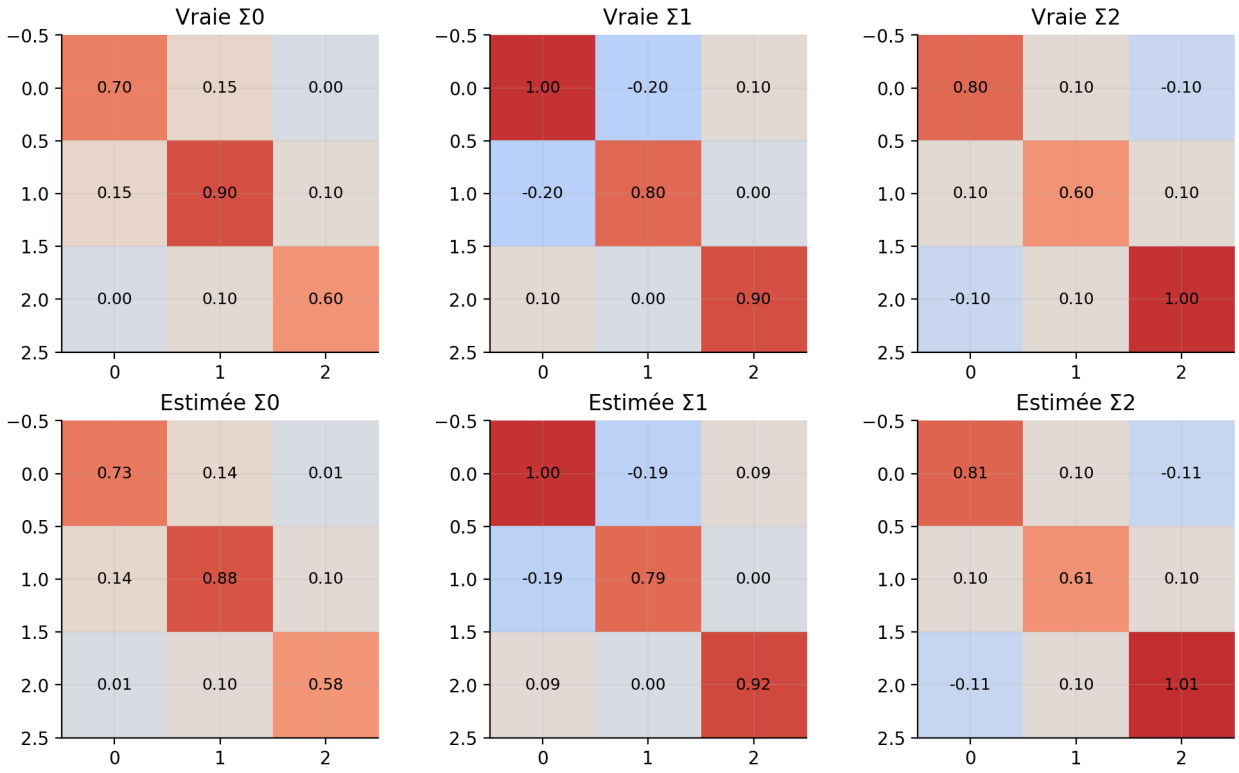
L'augmentation observée de la vraisemblance valide l'exécution sur cet exemple. EM peut atteindre des optima locaux ; les trois initialisations ne constituent pas une preuve de maximum global. Le graphique de droite est une projection x1/x2 d'un mélange défini dans trois dimensions.

6. GMM — alignement et comparaison des paramètres

6.1. Label switching

Permuter les indices des composantes ne change pas la densité du mélange. Comparer « composante 0 » à « composante 0 » sans alignement peut donner une erreur artificielle. Pour K=3, les six permutations sont testées ; le coût choisi est la somme des distances quadratiques entre moyennes vraies et estimées. La permutation retenue est [2, 1, 0].

Composante vraie	Poids vrai / estimé	Moyenne vraie / estimée
0	0.2500 / 0.2520	[-4.0000 ; 0.0000 ; 2.0000] [-3.9903 ; -0.0476 ; 2.0091]
1	0.4500 / 0.4470	[0.0000 ; 4.0000 ; -2.0000] [-0.0586 ; 3.9870 ; -2.0234]
2	0.3000 / 0.3010	[4.0000 ; -3.0000 ; 0.0000] [4.0111 ; -3.0162 ; 0.0297]

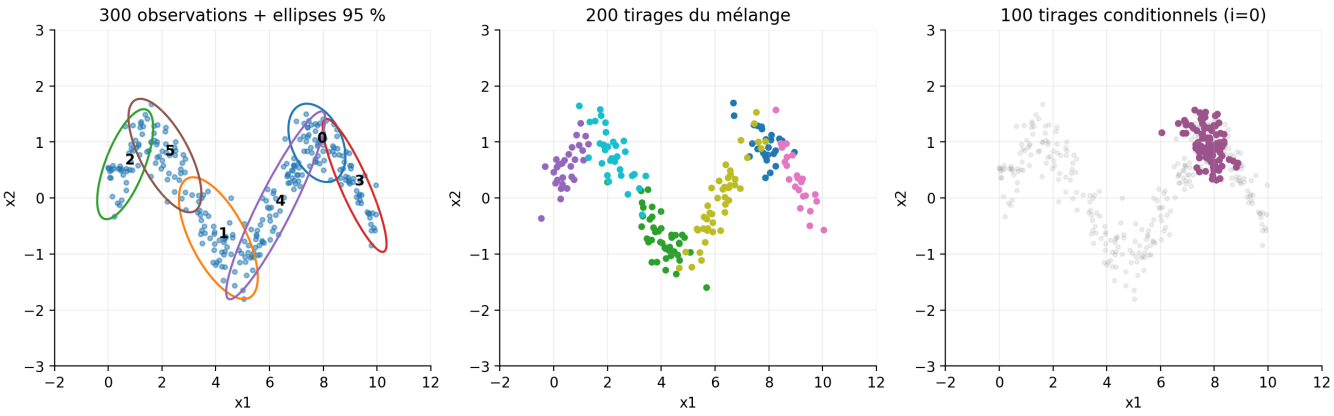


Les covariances comparées sont elles aussi réordonnées selon la permutation choisie. Les écarts restants sont de faibles variations d'estimation sur les observations simulées. Toutes les matrices vraies et estimées sont enregistrées sans arrondi dans metriques_partie1.json ; les heatmaps affichent deux décimales pour faciliter la lecture.

L'alignement par proximité des moyennes est suffisant pour ces composantes séparées. Des composantes presque identiques exigeraient un critère plus informatif prenant aussi en compte covariance ou distance entre distributions.

7. GMM sinusoïdal — génération globale et conditionnelle

Le nuage comporte 300 points : x1 uniforme dans [0,10], x2=sin(x1)+ε, avec ε gaussien centré de variance 0,1. L'écart-type utilisé est donc √0,1≈0,31623. GaussianMixture ajuste six composantes à covariance complète, avec trois initialisations et graine 42.



Le modèle a convergé en 9 itérations. Les ellipses représentent des contours conditionnels de masse 95 % pour chacune des six gaussiennes 2D ; leur rayon de Mahalanobis est √(−2log 0,05). Elles ne constituent pas une seule région de confiance globale du mélange.

7.1. Tirages depuis le mélange

Les 200 points globaux suivent la distribution apprise : une composante est tirée selon son poids, puis un point selon sa gaussienne. Les couleurs indiquent les indices tirés. Cette génération approxime plusieurs segments de la courbe, sans reproduire exactement sa loi initiale.

7.2. Tirages conditionnés sur i=0

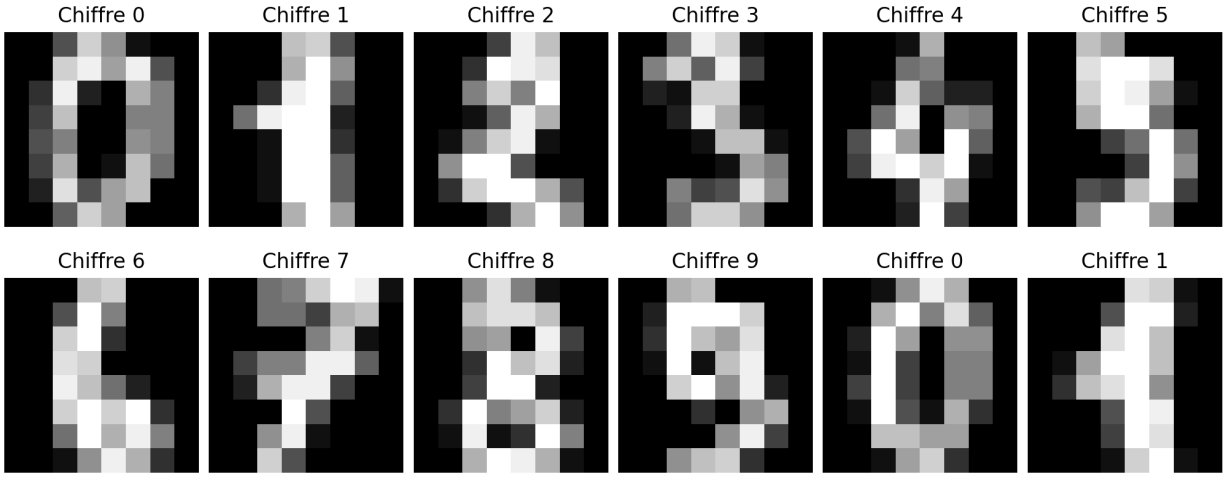
Les 100 points conditionnels suivent seulement N(μ0,Σ0). Leur nuage reste localisé autour d'une composante. Fixer i enlève la diversité provenant du choix de composante, mais conserve l'aléa gaussien interne. Le numéro 0 est un indice du modèle ajusté, sans signification métier universelle.

7.3. Limites

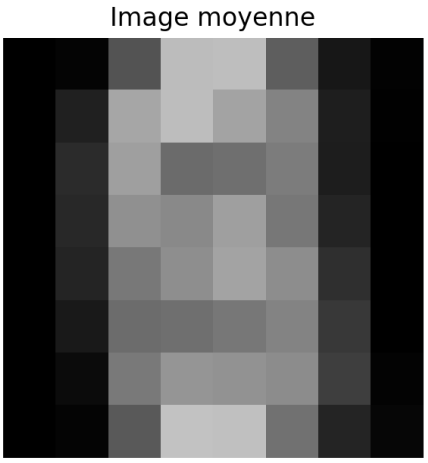
Une gaussienne possède un support non borné ; le GMM peut donc générer des x1 hors [0,10]. Les ellipses locales approchent la courbure mais ne garantissent pas que tous les points générés respectent la relation sinusoïdale. La convergence de l'algorithme ne garantit pas le bon nombre de composantes ni un modèle parfaitement fidèle.

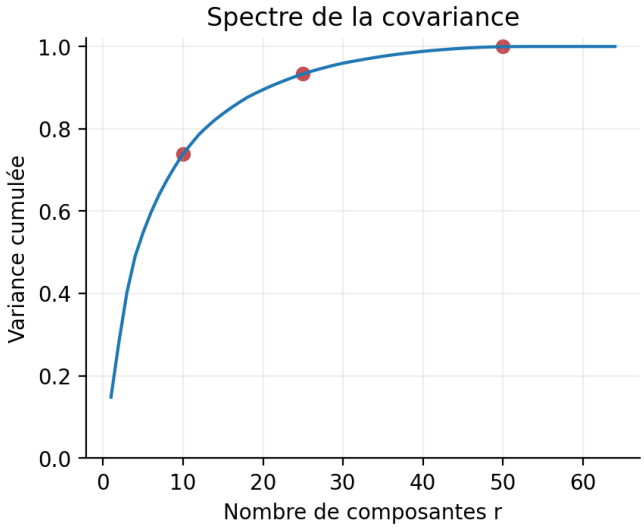
8. Images — données, moyenne et covariance

Le sujet autorise un jeu d'images grayscale. Le choix est load_digits, disponible localement : 1 797 images 8×8, intensités 0 à 16. Les 1 500 premières images servent à ajuster le modèle, après division par 16. Chaque image devient un vecteur de dimension 64. Les 297 autres ne sont pas utilisées pour l'apprentissage ni pour une évaluation de généralisation. Source du jeu.



La moyenne est la moyenne des vecteurs. La covariance suit exactement le sujet : $(X-\mu)^T(X-\mu)/N$. Cette matrice encode la variation conjointe des pixels. Les contrôles établissent sa symétrie et sa semi-définie positivité à la tolérance numérique.

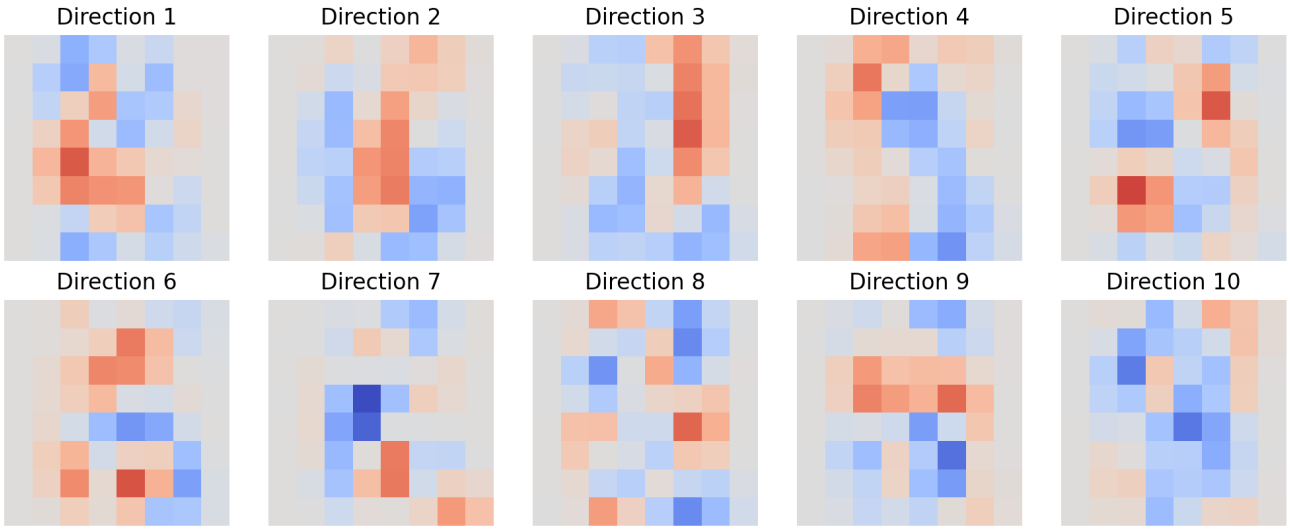




L'image moyenne mélange les traits de plusieurs classes de chiffres et n'est pas censée être une nouvelle observation réaliste. La courbe à droite décrit l'énergie de variation conservée par les premières directions. La covariance a un rang numérique de 61 sur 64 : des pixels constants conduisent à des variances nulles. Une covariance empirique d'images n'est donc pas forcément strictement définie positive.

9. ACP — eigen-images et choix du rang

La décomposition spectrale est $\Sigma = E\Lambda E^T$. Les valeurs propres sont classées par ordre décroissant ; les colonnes de E sont les directions principales. Les très faibles valeurs propres négatives dues à l'arrondi sont ramenées à zéro après avoir contrôlé qu'elles sont supérieures à -10^{-10}. La reconstruction $E\Lambda E^T$ concorde avec la covariance.



Une eigen-image est une direction signée, pas une image ordinaire. Rouge et bleu indiquent l'augmentation ou la diminution d'intensité lorsqu'on varie la coordonnée correspondante. La palette est centrée sur zéro avec une échelle commune. Le signe d'un vecteur propre est arbitraire.

Rang r	Variance cumulée conservée
10	73.9196 %
25	93.3613 %
50	99.9511 %

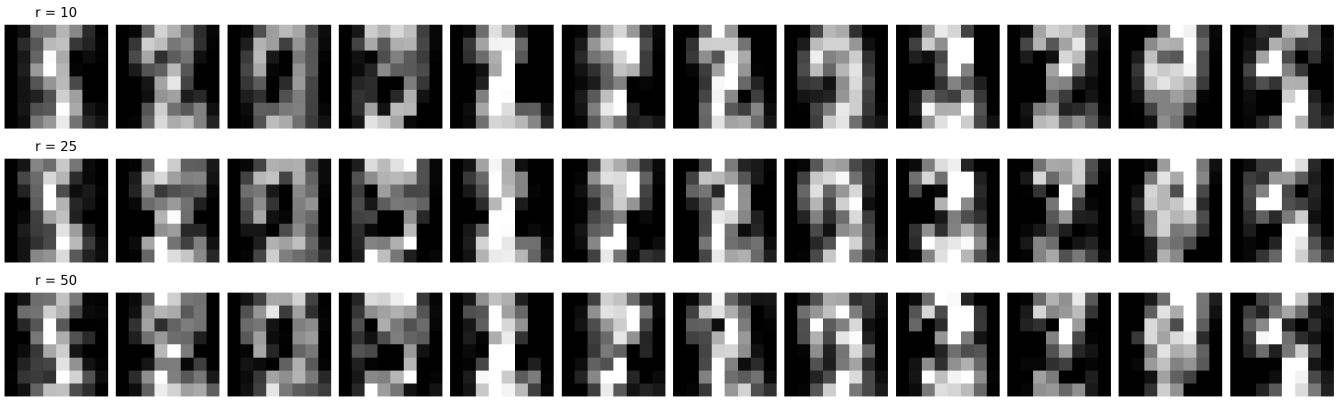
Dix directions représentent déjà les variations principales, mais perdent une partie de la covariance. Vingt-cinq conservent environ 93,36 %. Cinquante couvrent presque toute la variance. Ce pourcentage mesure une reconstruction de second ordre ; il n'est pas un score de réalisme des échantillons.

9.1. Paramètres du générateur tronqué

$A_r = E_r\,\mathrm{diag}(\sqrt{\lambda_1},\ldots,\sqrt{\lambda_r})$, de taille 64×r. Les tirages utilisent $z \sim N(0, I_r)$, puis $x = \mu + A_r z$. Le générateur produit une gaussienne sur un sous-espace affine de rang au plus r. Si r<64, sa covariance est singulière dans l'espace complet : il s'agit bien d'un modèle de faible rang, pas d'une gaussienne à covariance inversible sur tous les pixels.

10. Images — comparer r=10,25,50

Douze vecteurs latents de 50 coordonnées sont tirés. Les essais r=10 et r=25 réutilisent leurs premières coordonnées. Cette comparaison contrôlée isole l’ajout de directions plutôt que de changer entièrement les réalisations aléatoires.



Les trois lignes correspondent aux trois rangs. Les sorties montrent des variations de traits autour de la moyenne, parfois proches d’un chiffre, souvent avec superposition. Ajouter des directions enrichit les détails ; cela ne garantit pas une image plus cohérente ni une classe reconnaissable.

Rang	Pixels bruts hors [0,1] dans le lot de 12
10	19.66 %
25	21.88 %
50	25.26 %

Le taux ci-dessus utilise le test strict raw<0 ou raw>1, y compris de très faibles écarts numériques. Le clipping à [0,1] sert uniquement à l’affichage. Les sorties brutes et les coordonnées latentes sont conservées dans modele_images.npz ; le clipping modifie la loi et ne doit pas être présenté comme faisant partie de la gaussienne d’origine.

10.1. Contrôle des moments avant clipping

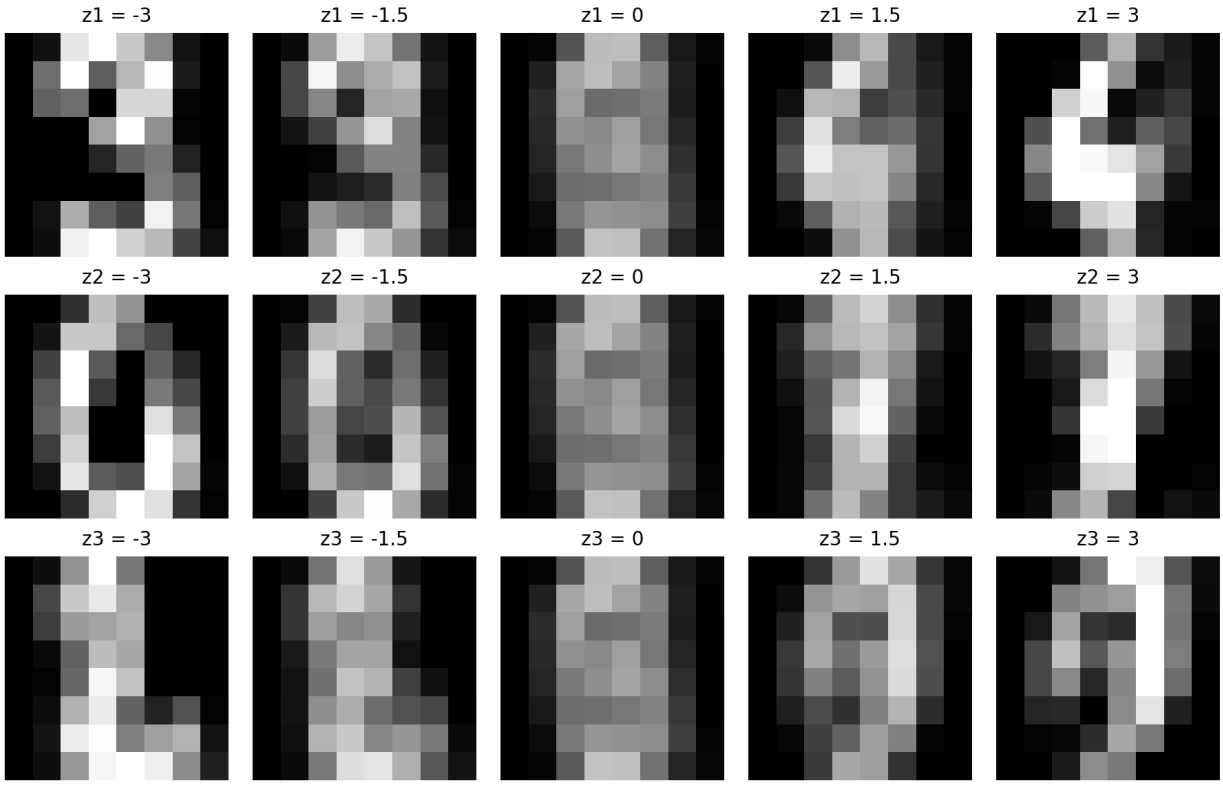
Pour r=25, un lot supplémentaire de 20 000 tirages contrôle les moments. L’erreur L2 de moyenne vaut 0.015286 et l’erreur relative de covariance vaut 0.020916. Ces erreurs Monte-Carlo sont inférieures aux seuils de 0,05 utilisés par le notebook. Elles ne sont pas des scores perceptuels.

Les checks vérifient aussi $AA^T = E_r \Lambda_r E_r^T$ algébriquement. Ils établissent que l’échantillonnage correspond au générateur gaussien annoncé, sans conclure que les images sont réalistes.

11. G(z) — interprétation des variables latentes

11.1. Un générateur affine

G(z)=μ+Az est affine. Sa partie Az est linéaire, mais la translation μ empêche de le qualifier de strictement linéaire lorsque μ≠0. Avec E[z]=0 et Cov[z]=I, on obtient E[G(z)]=μ et Cov[G(z)]=AA^T. La génération transforme ainsi une loi latente standard en une loi d'images.



Chaque ligne fait varier une seule des trois premières coordonnées : −3 ; −1,5 ; 0 ; 1,5 ; 3. Toutes les autres restent à zéro. Le centre est l'image moyenne. La variation brute vaut $z_j\sqrt{\lambda_j}\,e_j$; les premières directions ont les plus grandes amplitudes de variance. L'affichage est clippé, comme les autres images générées.

11.2. Interprétation et limites

Les directions dominantes déplacent ou renforcent des groupes de pixels et changent la forme apparente des traits. Elles ne représentent pas nécessairement une propriété sémantique unique, comme « inclinaison » ou « épaisseur ». Le mélange de classes peut distribuer plusieurs effets dans une même direction.

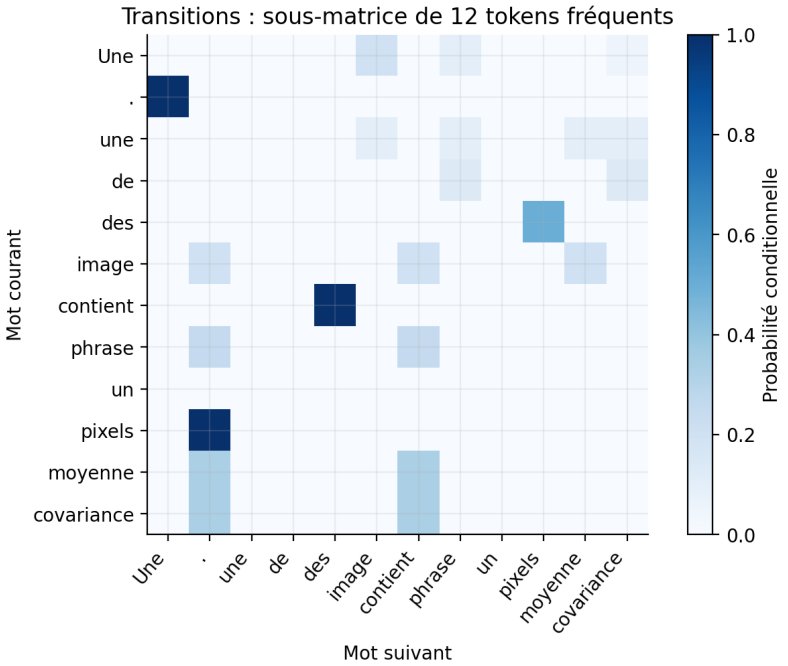
Le générateur n'a ni activation non linéaire ni partition explicite par classe. Une unique gaussienne reste unimodale et peut produire des interpolations de traits peu plausibles. Pour modéliser plusieurs modes, on pourrait étudier un mélange ou un générateur non linéaire ; ce TP n'entraîne ni GAN ni VAE.

12. Chaîne de Markov — corpus français

Le corpus local comporte 166 tokens et 70 états uniques. Un tokenizeur simple conserve mots accentués, apostrophes et points. make_pairs produit les paires consécutives ; build_transition_matrix stocke d[w1][w2]=occurrences. Les lignes sont normalisées au moment du tirage avec numpy.random.choice.

Bigramme	Occurrences
. → Une	19
Une → image	4
contient → des	4
des → pixels	3
pixels → .	3
chiffre → .	2
une → variation	2
loi → gaussienne	2
et → une	2
Une → direction	2

Le dictionnaire conserve 131 transitions observées contre 4900 cases pour une matrice dense. Le corpus est concaténé : le bigramme point → Une apparaît entre phrases. La génération s'arrête dès le point ; cette transition n'est donc pas utilisée après la fin de la phrase générée.



La heatmap montre une sous-matrice de 12 tokens fréquents : les lignes sont normalisées sur le vocabulaire complet, pas seulement sur les colonnes affichées. Phrase demandée : Une distribution. Elle commence par Une et s'est arrêtée sur le point. Ce résultat court respecte l'algorithme, sans fournir une preuve de cohérence linguistique.

13. Séquences symboliques — probabilités et NLL

13.1. Corpus exact et factorisation

```
(A, B, C, <eos>)    (A, B, D, <eos>)
(A, C, D, <eos>)    (B, C, D, <eos>)

p(x1,x2,x3,x4) = p(x1) p(x2|x1) p(x3|x1,x2)
                 p(x4|x1,x2,x3)
```

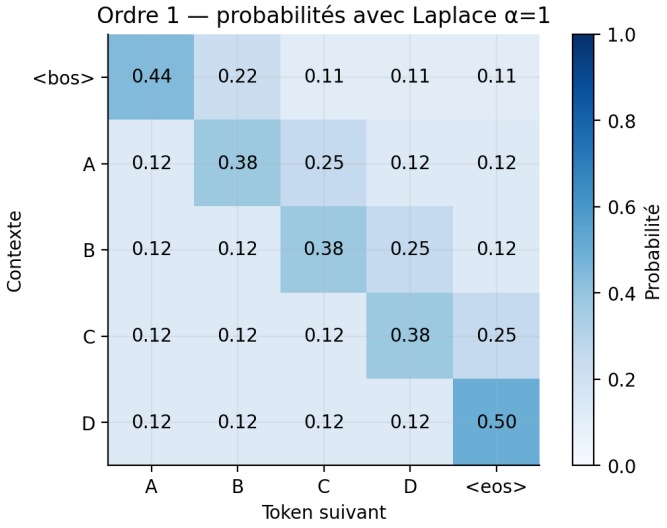
L'ordre 1 remplace chaque historique par le dernier token. Le vocabulaire prédit a cinq valeurs A, B, C, D et eos. Le token bos est ajouté comme contexte initial, pas comme sortie. La probabilité du premier token et celle de eos sont incluses dans la NLL.

Contexte	A	B	C	D	eos
bos	3/4	1/4	0	0	0
A	0	2/3	1/3	0	0
B	0	0	2/3	1/3	0
C	0	0	0	2/3	1/3
D	0	0	0	0	1

Pour (A,B,C,eos), la probabilité d'ordre 1 est (3/4)(2/3)(2/3)(1/3)=1/9, soit une NLL de log 9≈2,19722 nats. La NLL moyenne du corpus est la moyenne des quatre sommes ; la NLL par token divise le total par 16, puisque chaque séquence contient quatre tokens, fin comprise.

13.2. Lissage Laplace

Avec α=1, p(w|c)=(C(c,w)+1)/(Σv C(c,v)+5). Toutes les sorties deviennent possibles, y compris des transitions absentes de l'entraînement. Cela empêche des probabilités nulles dans un contexte nouveau, mais réduit la probabilité des transitions déjà observées.



NLL d'ordre 1 par séquence : 1.994492 sans lissage ; 4.015042 avec Laplace. La hausse sur l'entraînement est attendue : le MLE maximise la vraisemblance des observations déjà vues. Une évaluation hors entraînement serait nécessaire pour juger le bénéfice du lissage.

14. Génération — vingt séquences par variante

La génération commence par un contexte bos, tire chaque token selon la ligne conditionnelle, puis s'arrête sur eos. Une limite de 15 tokens protège des boucles en présence de lissage ; elle ne force pas artificiellement un token de fin. Le tableau ci-dessous présente les deux lots de vingt séquences d'ordre 1 effectivement tirées.

N°	Ordre 1 — MLE	Ordre 1 — Laplace α=1
1	A B C <eos>	C D <eos>
2	A C D <eos>	C A C D <eos>
3	A B D <eos>	A A D D D <eos>
4	B D <eos>	A B D <eos>
5	A C <eos>	<eos>
6	A B C <eos>	D C <eos>
7	A B C D <eos>	B A B D <eos>
8	A C <eos>	A C C B A A B A C <eos>
9	A B D <eos>	A C <eos>
10	A B D <eos>	D C D D A B A B <eos>
11	A B D <eos>	<eos>
12	B D <eos>	D <eos>
13	B D <eos>	<eos>
14	A B D <eos>	A C D B B D <eos>
15	A C D <eos>	C <eos>
16	B C D <eos>	A <eos>
17	A C D <eos>	B C <eos>
18	B C <eos>	A D B C D C D <eos>
19	A B C D <eos>	A A B A C D A D <eos>
20	A B C D <eos>	B A B C <eos>

Sans lissage : 8 séquences distinctes, 11 occurrences exactement présentes dans le corpus, 20/20 terminées. Avec lissage : 18 séquences distinctes, 1 occurrence présente dans le corpus, 20/20 terminées. Les quatre lots, dont les deux d'ordre 2, sont enregistrés dans sequences_generees.json.

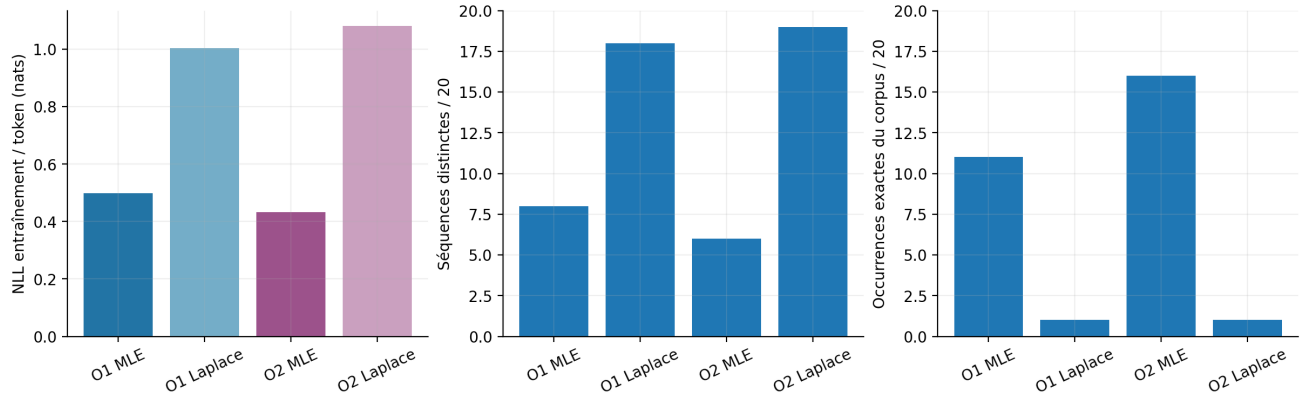
Laplace accroît ici la diversité et autorise des suites comme des retours ou des débuts atypiques. « Présente dans le corpus » mesure une correspondance exacte sur quatre exemples, pas une qualité grammaticale ni une préférence humaine. Des sorties inédites ne sont pas automatiquement mauvaises.

15. Ordre 2 — expressivité, sparsité et surapprentissage

15.1. Deux tokens de contexte

L'ordre 2 estime p(xt|xt−1,xt−2). Hors tokens spéciaux, une table complète demande V²(V−1) degrés de liberté contre V(V−1) à l'ordre 1. Ici, les contextes valides excluent eos comme état actif et incluent explicitement le début : cinq contextes possibles à l'ordre 1 ; 1+4+16=21 à l'ordre 2.

Structure	Ordre 1	Ordre 2
Contextes possibles	5	21
Contextes observés	5	8
Paramètres libres d'une table complète	20	84
Transitions stockées	9	12



Modèle	NLL / séquence	NLL / token	Uniques / 20	Corpus / 20
O1 MLE	1.994492	0.498623	8	11
O1 Laplace	4.015042	1.003761	18	1
O2 MLE	1.732868	0.433217	6	16
O2 Laplace	4.321576	1.080394	19	1

L'ordre 2 non lissé réduit la NLL d'entraînement et reproduit plus souvent les quatre séquences apprises dans ce lot. Avec Laplace α=1, la dilution est plus forte dans les contextes peu observés et sa NLL d'entraînement dépasse celle de l'ordre 1 lissé. Seuls 8 des 21 contextes sont vus ; les autres reçoivent une loi uniforme avec lissage.

L'approfondissement français d'ordre 2 utilise aussi des trigrams et un recul vers l'ordre 1 si un contexte manque. Exemple tiré : Une image représente un chiffre. L'amélioration de certaines phrases est une observation qualitative, sans test linguistique. Plus d'expressivité exige plus de données ; une moindre NLL sur quatre exemples ne prouve pas la généralisation.

16. Synthèse, limites et reproduction

16.1. Ce que les expériences établissent

La descente de gradient estime correctement les lois simples, à condition de respecter leurs domaines. Cholesky fournit une covariance valide et un calcul stable. EM apprend les composantes d'un mélange ; leur comparaison nécessite un alignement. Le générateur affine reproduit des moments empiriques d'images, mais pas leur structure sémantique. L'autorégression génère des séquences en combinant probabilités locales, avec un compromis entre mémoire, lissage et taille des données.

Contrôle	Résultat observé
Bernoulli	Deux méthodes à moins de 1e−6 du MLE ; θ toujours dans (0,1).
Gaussiennes	σ positif ; covariance 3D définie positive ; concordance avec MLE.
EM	Responsabilités normalisées ; poids positifs de somme 1 ; gain de vraisemblance non négatif sur l'essai retenu.
Images	Covariance /N et reconstruction spectrale ; génération 10/25/50 ; contrôle Monte-Carlo avant clipping.
Séquences	Transitions normalisées ; NLL finie du corpus ; 20 générations pour chaque couple ordre/lissage.
Notebooks	3 notebooks, 17 cellules de code exécutées sans sortie error, 15 figures.

16.2. Limites de l'évaluation

Les données probabilistes sont synthétiques et les paramètres sont connus par construction. Les images sont de faible résolution et le modèle n'est pas évalué sur le sous-ensemble réservé. Les métriques de séquences portent sur quatre exemples et des lots de vingt tirages ; elles ne valent pas mesures générales de qualité. Les contrôles prouvent l'exécution et les propriétés vérifiées, pas le réalisme universel ni l'absence de tout problème sur d'autres données.

16.3. Reproduction

Lancer Lancer_Notebooks.ps1, puis ouvrir les trois notebooks et choisir le kernel Python 3.12 (TP modèles génératifs). Exécuter chaque notebook dans l'ordre. Les cellules enregistrent leurs figures et métriques dans le dossier courant. Les dépendances préexistantes sont exposées par system-site-packages ; requirements_portables.txt liste des contraintes de versions stables pour une installation neuve à vérifier. L'instantané exact, dont PyTorch de développement, est fourni séparément.

16.4. Sources

Sujet fourni : Introduction aux modèles génératifs, [redacted], 20 avril 2026, huit pages.
PyTorch — softplus
scikit-learn — load_digits
scikit-learn — GaussianMixture et sample
Sources officielles consultées le 6 octobre 2026. Valeurs numériques : JSON et sorties des notebooks exécutés joints.